

CONTEXT ENGINEERING HANDS-ON

O'Reilly Live Training — Design, manage, and optimize the context that flows into LLMs and agentic systems

5 Sessions · 4 Runnable Demos · 1 Notebook | Python 3.12+ · uv · Anthropic Claude API | Lucas Soares

1. THE FOUR LEVERS — WRITE · SELECT · COMPRESS · ISOLATE

WRITE — Create

- Plans & scratchpads during a task
- Learned memories across sessions
- Filesystem logs & progress files
- Write claims + sources to verify

→ *todo.md*, *plan files*, *JSON logs*

SELECT — Pull

- Agentic retrieval (grep, glob, fetch)
- Tool loadout: only what's needed
- Progressive disclosure vs. stuffing
- *llm.txt* + fetch tool (Lance Martin)

→ *Claude Code*: no indexing, pure agentic retrieval

COMPRESS — Shrink

- **Offload to disk first** (lossy safety net)
- Summarize with high recall (exhaustive bullets)
- Auto-compact at 95% capacity (Claude Code)
- Trim verbose tool outputs after processing
- Cache invariant context (system prompts)

ISOLATE — Separate

- Sub-agents: clean window, summary return
- Sandbox: persists state (images, data files)
- State objects (Pydantic): expose fields selectively per stage
- Read tasks parallelize — write tasks don't

2. FOUR FAILURE MODES + FIXES (DREW BREUNIG'S TAXONOMY)

POISONING — Hallucination compounds

A bad output enters context and is repeatedly referenced as ground truth. Agent becomes fixated on impossible goals (Gemini Plays Pokémon).

FIX: ISOLATE + SELECT

- Keep raw tool outputs in **separate state fields**
- Promote only **validated facts** into next prompt
- Write a scratchpad of claims + sources; forward verified only

DISTRACTION — Model ignores training

Context grows so long the model over-focuses on it. Gemini 2.5 Pro: ↓ performance beyond **100K tokens**; defaulted to repeating past actions. Llama 3.1 405B ceiling: ~32K.

FIX: COMPRESS + SELECT

- Periodically **summarize history**; reinitiate with summary
- Pull smaller working set each step
- Write persistent plan so you can compress without losing intent

CONFUSION — Irrelevant content pollutes

If you put it in context, the model attends to it. Berkeley: Llama 3.1 8B **failed** with 46 tools — **succeeded** with 19. Every model worsens with more tools.

FIX: SELECT + ISOLATE

- **Dynamic tool loading** (LLM-powered recommender: **+44%** Llama 3.1 8B)
- Keep "nice-to-have" context in separate state fields
- DeepSeek-v3 fails at 100+ tools — keep loadouts small

CLASH — Conflicting context compounds

New info conflicts with earlier context. Microsoft/Salesforce: **39% avg drop** when identical info spread across turns. o3: 98.1 → 64.1. Wrong turns don't self-correct.

FIX: ISOLATE + SELECT

- **Context quarantine**: multi-agent isolated context → +90.2% on parallelizable tasks
- Apply **precedence rules** to decide what's included this step
- Compress conflicting sections into single resolved summary

3. CONTEXT PRIORITIZATION & DEBUGGING LOOP

Context Prioritization Tiers

- **CRITICAL** — current code, error messages, active task → *always include*
- **HELPFUL** — project structure, conventions → *if space permits*
- **OPTIONAL** — historical decisions, background → *only if needed*

"Find the smallest set of high-signal tokens that maximize the likelihood of your desired outcome."

— Anthropic, Effective Context Engineering for AI Agents

Models can ignore irrelevant info but cannot perform without relevant info — *prioritize recall.*

The Debugging Loop

1. **Inspect trace** — walk through messages, tool calls, and outputs turn-by-turn
2. **Tag failure** — which of the four modes? (Poisoning / Distraction / Confusion / Clash)
3. **Pick a lever** — W / S / C / I based on the mode (Select appears in every fix)
4. **Re-run & evaluate** — measure outcome quality, not just token count
5. **Iterate** — context engineering is cyclical, not one-shot

Context pruning reduced a Wikipedia article by 95% while preserving QA accuracy. Offload + domain prompts: up to +54% performance.

4. KEY STATS FROM PRODUCTION

46.9%

Token reduction — Cursor dynamic context discovery (MCP tools)

Cursor Engineering · 2026 A/B test

80% → 100%

Vercel removed 80% of tools (15 → 1 bash primitive); success rate went **up**

Vercel Engineering · Dec 2025

-3% / +20%

LLM-generated CLAUDE.md: -3% success rate, +20% inference cost

ETH Zurich · arXiv 2602.11988

16×

Context engineering is ~16× more impactful than model choice (80% of variance)

Koylan, Agent Skills eval

39% drop

Avg performance drop — identical info spread across turns vs. given at once. o3: 98.1 → 64.1

Microsoft/Salesforce multi-turn

+44%

LLM-powered tool recommender improved Llama 3.1 8B on GeoEngine

Chroma Research / Drew Breunig

83.9%

Tool outputs consume 83.9% of total context in typical agent trajectories

philschmid.de / Manus analysis

500K / \$1–2

Tokens per run & cost for naive deep-research agent

Lance Martin (LangChain)

5. CLAUDE CODE PRIMITIVES → LEVER MAPPING

Primitive → Lever

CLAUDE.md	→ WRITE — Minimal static context; routing table not content dump. Human-written, <150–200 instructions.
Slash commands	→ WRITE — Reusable prompts authored once. /audit-context / ci_commit /review
Hooks	→ COMPRESS + ISOLATE — post-tool-use, pre-commit, session-start. Sidecar files without changing prompts.
Sub-agents	→ ISOLATE — Clean window, scoped toolset, summary- only return to parent.
MCP servers	→ SELECT — Composable context primitives, vendor- agnostic. Use fully qualified names (Server:tool_name).

CLAUDE.md — What Works

ETH Zurich (arXiv 2602.11988): agents follow CLAUDE.md and use mentioned tools 1.6–2.5× more — but this costs 14–22% more reasoning tokens without solving more tasks.

✓ Put IN:

- Project purpose (one line)
- Non-obvious commands (build, test, deploy)
- Hard-to-infer conventions & package manager quirks

✗ Keep OUT:

- Anything already in README or code comments
- Anything the agent can ls to discover
- LLM-generated content (–3% success, +20% cost)

Claude Code system prompt already uses ~50 instructions — budget the remaining ~100–150 carefully.

6. PATTERN SELECTION GUIDE

When to Use Which Pattern

Situation	Pattern	Why
Stable project conventions	Minimal CLAUDE.md	Rarely changes; static is fine
Topic-specific research	Context priming at session start	Fetches current docs, not stale weights
Large codebases, many tools	Dynamic discovery + tool consolidation	Avoids Confusion; 46.9% token savings (Cursor)
Long-running multi-step agents	Tiered architecture + filesystem	Hot/cold separation; pre-rot at 70–80%
Multi-agent systems	Communication over shared context	"Share memory by communicating" — Peak Ji (Manus)
Repetitive learnable workflows	Self-improving contexts (ACE)	No manual maintenance; contexts evolve from execution

7. QUICK START — 5 SESSIONS · 4 DEMOS · 1 NOTEBOOK

Setup

```
# Install uv (once)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Set API key
export ANTHROPIC_API_KEY=sk-ant-...

# Run any demo
cd demos/<name>
uv run app.py
# No global pip install needed
```

Session 1 & 2 Demos

```
# S1: Claude Code live demos
# (no script — follow guide)
demos/ctx-engineering-principles-claude-code/live-demo-guide.md

# S2: Hand-rolled agent loop
cd demos/agent-ic-retrieval
uv run app.py
# Slash commands: /context
# /stats /clear /docs
```

Session 2/4 & 3 Demos

```
# S2/4: FastAPI chat app
cd demos/chat-with-artifacts
uv run app.py
# Open: http://127.0.0.1:8000

# S3: Context failures notebook
# demos/context-failures/context_failures.ipynb
# 4 modes: Poisoning, Distraction, Confusion, Clash — broken + fixed
```

8. PRODUCTION LESSONS — MANUS FIVE STRATEGIES (PEAK JI)

Five Strategies for Production Context

1. **Compaction & summarization** — reversible (paths, not contents) + lossy at ~128K tokens; always offload before summarizing
2. **Communication over shared context** — fresh sub-agents for discrete tasks; never let agents share a growing context window
3. **Hierarchical action spaces** — ~20 atomic tools + sandbox utilities + code libraries for complex chains
4. **Agent-as-tool** — sub-agents as deterministic functions with JSON schemas; parent sees only structured output
5. **Pre-rot thresholds** — trigger compaction at **70–80% utilization**, proactively (not at overflow)

Token Economics at Scale

Architecture	Token multiplier
Single agent (chat)	1× baseline
Single agent + tools	~4× baseline
Multi-agent system	~15× baseline

"Share memory by communicating, don't communicate by sharing memory."

— Peak Ji, Manus

Context engineering ≈ 80% of performance variance. Tool calls ~10%. Model choice ~5%. Optimize what the agent *sees* before upgrading the model.